

NEXO · MIKROWISP

Documentación de la API

Referencia completa de endpoints, autenticación y webhooks

Documento fuente para construir un portal de documentación al estilo de SmartOLT API y Mikrowisp API Reference.

Instancia	https://vente.rsgve.com
Base URL	https://TU-DOMINIO
Núcleo	Mikrowisp 6.x + módulos NEXO
Familias	v1 / v11 · v2 / v21 · Auth JWT · Webhooks · Recaudadores
Formato	JSON UTF-8 (SOAP solo en BancoEstado)
Fecha	6 de octubre de 2026
Versión del documento	1.0

Índice

1. Cómo usar este documento (y cómo convertirlo en un portal)
2. Introducción y arquitectura
3. Autenticación
4. Convenciones, CORS y códigos HTTP
5. API v1 — Núcleo Mikrowisp
6. API v11 — Proxy Bearer de v1
7. API v2 / v21 — Módulos NEXO
8. Webhooks (notificaciones entrantes)
9. Recaudadores y pasarelas bancarias
10. Catálogo rápido de rutas
11. Errores frecuentes y checklist de integración

1. Cómo usar este documento

Este PDF es la fuente de verdad de todo lo que un integrador puede consumir en este servidor. Está escrito con el mismo nivel de detalle que un portal tipo **ReadMe / Mintlify / Redoc**, para que más adelante cada sección se convierta en una página del sitio de documentación.

1.1 Qué cubre y qué no

- **Sí:** APIs HTTP públicas o de integración (v1, v11, v2, v21, /api/auth), webhooks que llaman bancos y proveedores, y recaudadores (Facilito, Punto Ágil, Evertec, BancoEstado).
- **Sí:** parámetros, autenticación, ejemplos curl, respuestas JSON, efectos de negocio (pago, activación, RADIUS, Mikrotik).
- **No:** el panel admin, AJAX interno del navegador, cron, ni scripts de diagnóstico no pensados para terceros.

1.2 Cómo mapear esto a un portal (SmartOLT / Mikrowisp)

Cuando construyas la página, replica esta taxonomía. Cada bloque **Endpoint** de este PDF debe convertirse en una ficha con: método, ruta, descripción, auth, tabla de parámetros, ejemplo de request, ejemplo de response y errores.

Sección de este PDF	Página del portal
Cap. 2 Introducción	Home / Getting started
Cap. 3 Autenticación	Guía “Authentication”
Cap. 5 API v1	Grupo “Mikrowisp core” — un item por comando
Cap. 7 API v2	Grupo “NEXO” — un item por módulo/acción
Cap. 8 Webhooks	Grupo “Webhooks” (incoming)
Cap. 9 Recaudadores	Grupo “Collectors / Banks”

Recomendación: para integraciones nuevas usa /api/v21 (NEXO) y /api/v11 (Mikrowisp) con **Authorization: Bearer**. Evita meter el token en query string en producción.

2. Introducción y arquitectura

NEXO es la capa de gestión ISP construida sobre **Mikrowisp**. Expone dos mundos de API que conviven en el mismo dominio:

- **API v1** — núcleo oficial de Mikrowisp (clientes, facturas, tickets, servicios, SmartOLT). El archivo está ofuscado (ionCube); el contrato público es el de la documentación oficial de Mikrowisp, más el proxy v11 añadido por NEXO.
- **API v2** — módulos propios de NEXO (pago móvil, C2P BDV, hotspot, Social WiFi, NAP, sucursales, vehículos, métricas, IPTV, etc.).

2.1 Base URL

```
https://TU-DOMINIO
# Instancia actual:
https://vente.rsgve.com
```

Familia	Base	Uso
API v1	/api/v1/{Comando}	Núcleo Mikrowisp. POST JSON con campo token.
API v11	/api/v11/{Comando}	Mismos comandos que v1, acepta Bearer (JWT o token API).
API v2	/api/v2/{modulo}/{id accion}	REST NEXO. Token en query, body o header según módulo.
API v21	/api/v21/{modulo}/{id accion}	Igual que v2 + Bearer. Recomendada.
Auth	/api/auth	Emite y renueva JWT (1 hora).
Webhooks	/api/webhooks/{proveedor}	El banco/proveedor llama al servidor.
Recaudadores	/recaudador/{operacion}	Consulta de deuda, pago y reverso (Facilito, Punto Ágil, ...).

2.2 Diagrama

```

Cliente / App / Banco
|
+--> POST /api/auth          --> JWT (3600 s)
|
+--> Bearer JWT o token_api
      |
      +--> /api/v11/{cmd}  --proxy--> /api/v1/{cmd}  (Mikrowisp)
      +--> /api/v21/{mod}  --proxy--> /api/v2/{mod}  (NEXO)
|
+--> POST /api/webhooks/{banco} --> conciliación / addPago
+--> POST /facilito|puntoagil|evertec/... --> recaudación
```

v11 y v21 **no añaden lógica de negocio**: validan el Bearer, resuelven el token_api del operador y reenvían a v1/v2. Por eso la respuesta de v11 es idéntica a v1, y la de v21 es idéntica a v2.

2.3 Enrutamiento v2

Petición	Método PHP	Uso
GET /api/v2/{modulo}	getAll()	Listar
GET /api/v2/{modulo}/{id}	getOne(id)	Detalle o subrecurso
POST /api/v2/{modulo}	create()	Acción principal
POST /api/v2/{modulo}/{accion}	{accion}()	Acción nombrada (pago, registro, events...)
PUT /api/v2/{modulo}/{id}	update(id)	Actualizar
DELETE /api/v2/{modulo}/{id}	delete(id)	Eliminar (ver aviso Apache)
OPTIONS /api/v2/...	—	CORS preflight, 200

El segmento {modulo} se convierte en clase {Modulo}Controller (hotspotfichas → HotspotfichasController). Módulo inexistente: 404 {"message": "Not Found"}.

DELETE bloqueado. El .htaccess del servidor responde **403** a HEAD, TRACE, DELETE, TRACK y DEBUG. Los DELETE documentados no funcionan hasta que se habilite el método. Usa la alternativa POST/PUT que se indica en cada módulo (por ejemplo activo: 0 en hotspotwifi).

3. Autenticación

Hay tres formas de autenticarse. El usuario debe existir en la tabla login, estar activo (estado = 1) y, para token API, tener api = 1 y un token_api generado en **Ajustes → Gestión personal / Usuarios administradores → API**.

3.1 Token API en el cuerpo (legacy, v1 y varios v2)

```
curl -X POST https://TU-DOMINIO/api/v1/GetClientsDetails \
-H "Content-Type: application/json" \
-d '{"token":"TU_TOKEN_API","idcliente":1}'
```

3.2 Bearer con token API (v11 / v21)

```
curl -X POST https://TU-DOMINIO/api/v11/GetClientsDetails \
-H "Authorization: Bearer TU_TOKEN_API" \
-H "Content-Type: application/json" \
-d '{"idcliente":1}'
```

3.3 JWT (recomendado)

El JWT se firma HS256, dura **3600 segundos** y se puede renovar solo si todavía está vigente. El objeto user nunca incluye password, token_api ni secretos 2FA.

POST /api/auth

Obtener token JWT

Sin autenticación previa. Credenciales de un administrador Mikrowisp.

Campo	Dónde	Tipo	Req.	Descripción
username	body / query	string	Sí	Usuario administrador
password	body / query	string	Sí	Contraseña (se compara con hash en BD)

```
curl -X POST https://TU-DOMINIO/api/auth \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"TU_PASSWORD"}'
```

```
{
  "status": "success",
  "user": { "id": 1, "username": "admin", "nombre": "Administrador",
            "email": "admin@tu-dominio.com", "estado": 1 },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "Bearer",
  "expires_in": 3600,
  "expires_at": 1791334951
}
```

POST /api/auth?action=refresh

Renovar JWT

Requiere un JWT todavía válido en Authorization: Bearer o en el campo token del cuerpo. Un token expirado no se renueva: hay que hacer login otra vez.

```
curl -X POST "https://TU-DOMINIO/api/auth?action=refresh" \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIs..."
```

3.4 Cómo distinguen v11/v21 JWT vs token API

El proxy busca el valor, en este orden, en los encabezados Authorization, BearerToken y Bearer-Token. Si el valor contiene dos puntos (xxx.yyy.zzz) se trata como JWT; si no, como token API. El usuario JWT debe tener token_api configurado porque el proxy lo inyecta al reenviar a v1/v2.

3.5 Errores de autenticación

HTTP	Mensaje típico	Causa
400	Usuario y contraseña son requeridos.	Faltó username o password
401	Usuario o contraseña incorrectos	Credenciales inválidas o usuario inactivo
401	Token requerido. / Token expirado. / Token inválido.	Refresh o Bearer ausente/vencido
403	Usuario no autorizado. / Token Bearer no autorizado.	Usuario desactivado o API no habilitada
403	Usuario autorizado, pero no tiene token_api configurado.	JWT OK pero falta token API para el proxy
500	Dependencia JWT no disponible.	Falta firebase/php-jwt
502	Error conectando con API legacy	El proxy no pudo hablar con v1/v2

4. Convenciones, CORS y códigos HTTP

- **Content-Type:** application/json en peticiones con cuerpo. Algunos módulos aceptan también application/x-www-form-urlencoded.
- **Codificación:** UTF-8. Las respuestas no escapan Unicode (JSON_UNESCAPED_UNICODE).
- **Fechas:** YYYY-MM-DD o YYYY-MM-DD HH:MM:SS, zona horaria del servidor.
- **Montos:** decimales con punto (10.50). Varios módulos trabajan USD y VES (bolívars) usando la tasa de dolarprice.
- **CORS:** v2 envía Access-Control-Allow-Origin: *. El /api/.htaccess fija además origen https://plano.com.ve.
- **Chatbot:** envía chatbot=1 o "chatbot": true en pago móvil / C2P / ventas hotspot para recibir un objeto chatbot con pasos sugeridos y ejemplos.

Formatos de respuesta

Familia	Éxito	Error
API v1 / v11	{"estado":"exito", ...}	{"estado":"error","mensaje":"salida":"..."}
Muchos v2 NEXO	{"ok":true,"salida":"..."}	{"ok":false,"salida":"..."}
Pagos / C2P / planificación	{"estado":"exito" "ok"}	{"estado":"error","salida":"..."}
/api/auth	{"status":"success"}	{"status":"error","message":"..."}

Códigos HTTP

Código	Significado en este servidor
200	Operación exitosa (a veces también errores de negocio en JSON)
201	Recurso creado (telemetría de vehículos)
400	Parámetros faltantes o formato inválido
401	No autenticado
403	Sin permiso, API deshabilitada, o método bloqueado por Apache
404	Módulo, cliente, factura o recurso inexistente
405	Método HTTP no permitido para esa ruta
409	Conflicto: referencia ya usada, movimiento no concilia, OTP inválido
422	Datos sintácticamente válidos pero rechazados por regla de negocio
500	Error interno
502	Fallo al contactar v1/v2, banco o servicio externo
503	Pasarela o módulo no configurado / no migrado

5. API v1 — Núcleo Mikrowisp

Todos los comandos v1 son **POST** a `/api/v1/{Comando}` (también `/api/v11/{Comando}` con Bearer). Cuerpo JSON. El campo token es obligatorio en v1; en v11 se omite si ya va el Bearer.

El enrutado acepta letras, números y guiones (`[a-z-0-9]+`), sin distinguir mayúsculas. Usa el nombre exacto del comando (`GetClientsDetails`, `PaidInvoice`, ...).

5.1 Facturación

POST `/api/v1/CreateInvoice`

Crear factura de servicio

Genera una nueva factura de servicio del cliente. Requiere token, idcliente y vencimiento.

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idcliente	number	Sí	ID del cliente
vencimiento	string	Sí	YYYY-MM-DD

```
{"token":"TU_TOKEN","idcliente":6,"vencimiento":"2026-11-01"}
```

```
{"estado":"exito","idfactura":130,"mensaje":"Nueva factura de servicios creado vía API - Factura ID: 130"}
```

POST `/api/v1/CreateInvoiceLibre`

Crear factura libre

Crea una factura libre (no ligada al ciclo automático de servicio). Mismos principios: POST JSON + token. Consulta el portal oficial para ítems/desglose si tu flujo lo requiere.

POST `/api/v1/GetInvoices`

Lista de facturas

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
estado	number	No	0 = pagadas, 1 = no pagadas, 2 = anuladas; vacío = todas
idcliente	number	No	Filtrar por cliente
limit	number	No	Tamaño de página (ej. 25)
pagina	number	No	Offset de página
fechapago	string	No	YYYY-MM-DD cuando se pagó
formapago	string	No	Pasarela o forma de pago

POST `/api/v1/GetInvoice`

Datos de una factura

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idfactura	number	Sí	ID de la factura

POST /api/v1/GetPlantillasFacturacion**Plantillas de configuración de facturación**

Lista las plantillas generadas en el sistema. Solo requiere token. En NEXO también existe el equivalente REST GET /api/v2/plantillasfacturacion.

POST /api/v1/PromesaPago**Promesa de pago**

Registra una promesa sobre una factura. La fecha límite máxima documentada por Mikrowisp es 20 días.

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idfactura	number	Sí	Factura
fechalimite	string	Sí	YYYY-MM-DD
descripcion	string	No	Motivo de la promesa

```
{"estado":"exito","mensaje":"Promesa de pago registrado correctamente."}
```

POST /api/v1/PaidInvoice**Pagar factura (y activar si está suspendido)**

Registra el pago. Si el cliente estaba SUSPENDIDO y el pago cubre lo necesario, Mikrowisp puede reactivar el servicio.

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idfactura	number	Sí	Factura a pagar
pasarela	string	Sí	Nombre visible (Paypal, Pago móvil, Efectivo...)
cantidad	number	No	Si se omite, toma el total de la factura
comision	number	No	Comisión del cobro
idtransaccion	string	No	Referencia / nro. de operación
fechalimite	string	No	Fecha-hora del pago; default ahora
nota	string	No	Notas del pago
afip	boolean	No	Emisión electrónica Argentina (si está configurado)

```
{"token":"TU_TOKEN","idfactura":130,"pasarela":"Pago móvil",
"cantidad":20,"idtransaccion":"001234","nota":"Ref. 001234"}
```

```
{"estado":"exito","mensaje":"Pago registrado correctamente."}
```

POST /api/v1/DeleteInvoice**Eliminar factura no pagada**

Solo facturas en estado no pagado. Requiere token e identificador de factura (idfactura).

POST /api/v1/DeleteTransaccion**Eliminar un pago**

Revierte / elimina el pago de una factura. Úsalo con cuidado: puede volver a dejar al cliente en deuda y afectar el estado del servicio.

POST /api/v1/ReportesPago**Reportes de pago del portal cliente**

Lista los reportes que el abonado cargó desde el portal (comprobantes pendientes de conciliar).

POST /api/v1/BaseFacturas**Base / listado de facturas con filtros**

Listado amplio de facturas (exportes, conciliaciones). Filtros similares a GetInvoices (estado, cliente, fechas).

POST /api/v1/GetPaymentLink**Links o referencias de pasarelas**

Pasarelas admitidas por Mikrowisp: MercadoPago, Siro, CobroDigital, OxxoPay, Toku. La respuesta cambia según la pasarela (url_pago, ticket_pdf, codigo_barras, redirect_url).

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
pasarela	string	Sí	MercadoPago Siro CobroDigital OxxoPay Toku
idfactura	number	Sí	Factura

5.2 Clientes

POST /api/v1/NewUser**Registrar cliente**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
nombre	string	Sí	Nombre completo
cedula	string	No	Cédula / DNI / RIF
correo	string	No	Email
telefono	string	No	Fijo
movil	string	No	Celular
direccion_principal	string	No	Dirección

Puedes enviar campos personalizados del resumen de cliente como claves extra en el JSON (N_orden, Alias, ...).

```
{"estado":"exito","mensaje":"El cliente fué registrado correctamente.","idcliente":100}
```

POST /api/v1/GetClientsDetails**Datos de cliente (con servicios y facturación)**

Busca por idcliente, cedula, telefono o movil. Devuelve ficha, servicios de internet (PPP, IP, ONU, NAP), otros servicios y resumen de deuda.

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idcliente	number	No*	ID interno
cedula	string	No*	Identificación
telefono	string	No*	Fijo
movil	string	No*	Móvil

*Al menos un criterio de búsqueda además del token.

```
{
  "estado": "exito",
  "datos": [
    {
      "id": 1775,
      "nombre": "...",
      "estado": "ACTIVO",
      "cedula": "...",
      "movil": "...",
      "servicios": [
        {
          "id": 8002,
          "perfil": "...",
          "ip": "...",
          "pppuser": "...",
          "onu_sn": "..."
        }
      ],
      "facturacion": {
        "facturas_nopagadas": 2,
        "total_facturas": "40.00"
      }
    }
  ]
}
```

POST /api/v1/GetAllClients**Lista de clientes por estado**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
estado	number	No	0 = ACTIVOS, 1 = SUSPENDIDOS, 2 = RETIRADOS
limit	number	No	Default 25
pagina	number	No	Página

POST /api/v1/UpdateUser**Actualizar cliente**

Los cambios van dentro del objeto datos. Campos personalizados también se envían ahí.

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idcliente	number	Sí	Cliente a editar
datos.nombre	string	No	Nombre
datos.correo	string	No	Email
datos.telefono / movil	string	No	Teléfonos
datos.cedula	string	No	Identificación
datos.codigo	string	No	Contraseña del portal cliente
datos.direccion_principal	string	No	Dirección

```
{
  "estado": "exito",
  "mensaje": "Datos actualizados correctamente."
}
```

POST /api/v1/ChangeFacturacionConfig**Cambiar configuración de facturación del cliente**

Ajusta ciclo, plantilla, día de pago u otros parámetros de facturación asociados al abonado. Requiere token e idcliente más los campos de configuración definidos en el panel.

POST /api/v1/ActiveService**Activar cliente suspendido**

Reactiva los servicios cuando el estado es SUSPENDIDO (Mikrotik / RADIUS / OLT según el nodo).

```
{"token":"TU_TOKEN","idcliente":33}
```

```
{"estado":"exito","mensaje":"Cliente Activado correctamente."}
```

POST /api/v1/SuspendService**Suspender cliente activo**

```
{"token":"TU_TOKEN","idcliente":33}
```

```
{"estado":"exito","mensaje":"Cliente Suspendido correctamente."}
```

POST /api/v1/NewPreRegistro**Pre-registro de instalación**

Crea un pre-registro en el módulo de instalaciones (lead / prospecto) para que un técnico lo gestione.

POST /api/v1/ListInstall**Lista de instalaciones**

Devuelve instalaciones (pendientes, agendadas, ejecutadas) según filtros del módulo de instalaciones.

5.3 Tickets y SMS

POST /api/v1/NewTicket**Crear ticket de soporte**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idcliente	number	Sí	Cliente
dp	string/number	Sí	ID departamento (1 = Soporte técnico)
asunto	string	Sí	Asunto
fechavisita	string	Sí	YYYY-MM-DD
turno	string	Sí	MAÑANA TARDE
agendado	string	Sí	VIA TELEFONICA PRESENCIAL PAGINA WEB RED SOCIAL
solicitante	string	No	Si no es el mismo cliente
contenido	string	No	Detalle (acepta HTML)
adjunto.nombre / file	object	No	Archivo en base64
propiedad	number	No	0 ninguno, 1 BAJA, 2 MEDIA, 3 ALTA
idtecnico	number	No	Asignar a un operador

```
{"estado":"exito","idticket":"100","mensaje":"Ticket Registrado correctamente."}
```

POST /api/v1/ListTicket**Listar tickets de un cliente**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idcliente	number	Sí	Cliente
idsorte	number	No	Un ticket concreto

La respuesta incluye contadores (abiertos, cerrados, respondidos) y el array `data.tickets`.

POST /api/v1/CloseTicket**Cerrar ticket**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
idticket	number	Sí	Ticket
motivo_cierre	string	No	Motivo

```
{"estado":"exito","mensaje":"Ticket cerrado correctamente."}
```

POST /api/v1/NewSMS**Enviar SMS**

Encola un SMS al cliente. Mikrowisp documenta que el envío ocurre aproximadamente un minuto después de la orden. Requiere token y datos del destinatario/mensaje configurados en el módulo SMS.

5.4 Red, servicios y OLT

POST /api/v1/GetRouters**Lista de routers (o uno por ID)**

Recupera nodos Mikrotik / routers registrados. Filtro opcional por ID.

POST /api/v1/GetMonitoreo**Equipos en monitoreo**

Lista equipos (APs, antenas, etc.) del módulo de monitoreo.

POST /api/v1/GetMacFromIp**Obtener MAC a partir de una IPv4****POST** /api/v1/GetRedesIpv4**Rangos IPv4 registrados****POST** /api/v1/GetIpv6FromDuid**Obtener IPv6 mediante DUID**

POST /api/v1/NewService**Crear servicio de internet**

Campo	Tipo	Req.	Descripción
token	string	Sí	Token API
id_cliente	number	Sí	Cliente
id_router	number	Sí	Nodo / Mikrotik
id_perfil	number	Sí	Plan
id_red_ipv4	number	Sí	Red IPv4
ipv4	string[]	Cond.	Obligatorio si la red es estática
mac	string	No	Única
userppp / passppp	string	No	Si se omiten, Mikrowisp los genera
caja_nap.id_caja / puerto	object	No	Asignación NAP
coordenadas, direccion, costo, ipv6...	—	No	Opcionales de instalación
equipo_receptor	object	No	Antena: id, ip, tipo 0-4, comunidad, user/pass

```
{"code": "200", "servicio_id": 10045, "mensaje": "Servicio registrado correctamente."}
```

POST /api/v1/EditService**Editar servicio de internet**

Modifica plan, IP, PPP, NAP u otros campos del servicio existente. Requiere token e identificador de servicio / cliente según el contrato oficial.

POST /api/v1/GetCajasNap**Cajas NAP (una o todas)**

Los filtros son acumulativos: id, descripcion, puertos, ubicacion, coordenadas, limit.

```
{"code": "200", "mensaje": "Operación exitosa", "datos": [{"id": 1, "descripcion": "CAJA NAP PRINCIPAL", "puertos": 16, "coordenadas": "-12.03, -76.93", "resumen_puertos": {"total": 16, "usados": 2, "libres": 14}}]}
```

POST /api/v1/GetOperadores · /GetDepartamentos · /GetTareas · /CreateTarea**Organización y tareas de campo**

Listan operadores, departamentos y tareas, o crean una tarea nueva. Todas requieren token. CreateTarea espera los campos del módulo de tareas (operador, descripción, fechas).

SmartOLT (vía Mikrowisp)

Si SmartOLT está integrado en el panel, estos comandos leen catálogos y autorizan ONU. Equivalente conceptual a consumir <https://api.smartolt.com/> pero a través de Mikrowisp.

Comando	Propósito	Params clave
SmartOltGetVlans	VLANs registradas	token
SmartOltGetZonas	Zonas OLT	token
SmartOltGetProfiles	Profiles de velocidad	token
SmartOltGetODB	ODBs	token
SmartOltAuthorizeONU	Autorizar ONU por id de servicio	token, id_servicio, sn, vlan_id, profile_up, profile_down; opc. onu_mode (Bridging Routing), modo_operacion (0 internet / 1 internet+CATV), zone, odb, dns1/dns2

```
{"code": "200", "mensaje": "Onu autorizado correctamente"}
```

6. API v11 — Proxy Bearer de v1

Rutas: /api/v11 o /api/v11/{Comando}. El sufijo se reenvía a `http://localhost/api/v1/{Comando}`. Inyecta token en el JSON. Errores del proxy (no de Mikrowisp) usan `{"estado":"error","salida":"..."}` con 401/403/500/502.

Para el portal de documentación, puedes listar **una sola vez** cada comando v1 y anotar: “también disponible en /api/v11/{Comando} con Bearer”.

7. API v2 / v21 — Módulos NEXO

Base: /api/v2/{modulo} y /api/v21/{modulo}. Salvo que se indique lo contrario, autentica con token API (query ?token=, body o Authorization: Bearer). v21 acepta JWT o token API.

Módulos registrados: facturacion, pagos, consultafactura, promociones, pagomovil, pagomovilbdv, bdvc2p, c2p, pagoc2p, planificacionareas, dolarprice, planes, metrics, hotspotfichas, hotspotwifi, hotspotventas, hotspotpartido, socialwifi, socialwifiregistro, sucursales, ubicaciones, nap, vehiculos, plantillasfacturacion, iptvsetplex.

UsuarioController existe como stub y **no está cableado** en el router: no es invocable.

7.1 Autenticación JWT (también vía AuthController en v2)

El login JWT **no** pasa por el router v2: usa /api/auth (capítulo 3). Si llamas /api/v2/auth, getAll/getOne/update/delete responden 405.

7.2 Consulta de deuda y pagos

GET /api/v2/consultafactura · /api/v2/consultafactura/{cedula}

Consulta pública de deuda por cédula

No exige token en el código actual. Pensada para portales / chatbots. Sin cédula responde que el servicio está activo.

```
GET /api/v2/consultafactura?cedula=V12345678
```

```
{
  "estado": "exito",
  "cliente": "Nombre",
  "cedula_cliente": "V12345678",
  "pasarela": { "banco": "...", "telefono": "...", "cedula": "...", "tasa_dia": 36.5 },
  "conteo": 1,
  "resultados": [{
    "id_factura": 100, "nfactura_token": "...",
    "monto_usd": 10.0, "monto_bs": 365.0,
    "vencimiento": "2026-06-01", "descripcion_sugerida": "..."
  }]
}
```

404 si no existe el usuario. 500 error interno.

POST /api/v2/facturacion**Registrar pagos de una o varias facturas**

Llama a `ModulosMikrowisp->addPago()`. Valida montos y tasa dolarprice de la fecha de pago. **El controlador no valida token_api**: protéjelo por red o usa v21 detrás de un gateway.

GET /api/v2/facturacion responde texto plano working.

Campo	Tipo	Req.	Descripción
idsFacturas	array	Sí	IDs en tabla facturas
moneda	string	Sí	VEF/VEB/VED o USD
amount	number	Sí	Debe coincidir con la suma (± 0.01)
transactionId	string	Sí	Referencia externa
formaPago	string	Sí	Etiqueta de pasarela
descripcion	string	Sí	Descripción
fechaPago	string	Sí	Para buscar la tasa
idcoin, comision	—	No	Moneda interna / comisión
pmtel, pmced, pmBancoOrigen	string	No	Metadata pago móvil
conciliarPagoMovil	bool	No	Intentar conciliación

```
{"success":true,"message":"Facturación agregada correctamente","facturasPagadas":2}
```

POST /api/v2/pagos**Conciliar pago móvil (legacy)**

Busca el movimiento en `pagos_bdv` (referencia ≥ 6 dígitos + monto + fecha) y registra el pago. Variante anterior de `pagomovil`.

Campo	Req.
token	Sí (body)
nfactura	Sí
referencia	Sí (≥ 6 dígitos)
monto	Sí
fecha	No (hoy)

401 token inválido. Errores de negocio suelen ir en HTTP 200 con estado: error (factura inexistente, referencia no hallada, ya validada, monto distinto).

GET / **POST** /api/v2/pagomovil**Consulta de deuda y conciliación de pago móvil**

Consulta por nfactura (id, lista o token de portal) o cedula/documento. POST sin referencia/monto = consulta. POST con referencia + monto = concilia contra pagos_bdv (últimos 6 dígitos + monto + fecha) y ejecuta addPago. Puede reactivar el servicio.

Campo	Req.	Notas
token / Bearer	Sí	Operador API
nfactura o cedula	Sí*	Uno de los dos en consulta
referencia, monto	Sí en pago	
fecha / pm_fecha	No	
pm_tel, cedula, pm_banco_origen, descripcion, chatbot	No	

```
{
  "estado": "exito",
  "ids_facturas": [10],
  "idcliente": 5,
  "monto_total_moneda_factura": 10,
  "monto_total_bs": 365,
  "facturas_pendientes": [],
  "saldo_otros_no_cobrado": 0
}
```

409 si no concilia. getOne → 404. update/delete → 405.

GET / **POST** /api/v2/pagomovilbdv**Pago móvil validado en vivo con API BDV**

Misma consulta que pagomovil. El POST valida el movimiento con getMovement de Banco de Venezuela (pasarela bancodevenezuela). **Una sola factura por request.**

Campo extra en POST	Req.
pm_ced / cedula	Sí
pm_tel	Sí (≥10 dígitos)
pm_banco_origen	Sí (≤4 dígitos)

400 si hay más de una factura. 503 pasarela inactiva. 409 el banco no valida el movimiento.

GET / **POST** /api/v2/c2p y /api/v2/bdvc2p**Pago C2P Banco de Venezuela**

Alias del mismo flujo. Pagador solo banco **0102**. GET/POST resumen con token de factura de portal. POST con accion:

accion	Qué hace	Campos
(vacío) / resumen	Igual que GET	token, nfactura
request_otp	Pide OTP al BDV	pagador_id, telefono_pagador
c2p_pay	Debita y registra pago	banco=0102, pagador_id, telefono_pagador, otp_c2p

```
{
  "estado": "exito",
  "pasarela_activa": true,
  "ids_facturas": [1,2],
  "monto_bs": 100,
  "comision_bs": 2,
  "monto_total_bs": 102,
  "banco_pagador_c2p_bdv": "0102",
  "tiene_telefono_cobrador": true
}
```

403 pasarela inactiva. 409 validación OTP/banco. 503 falta X-API-Key BDV. Efectos de c2p_pay: addPago, ajuste de bolívares en facturas USD, autorización Cashea si aplica.

GET / **POST** /api/v2/pagoc2p**Consulta estilo pagomovil + flujo C2P**

Permite consultar por nfactura o cédula (incluye montos C2P) y luego request_otp / c2p_pay. Rechaza un POST que solo traiga referencia+monto: eso va a pagomovil.

7.3 Catálogo, tasas y métricas

GET /api/v2/dolarprice · /api/v2/dolarprice/{id}

Tasa del dólar

Auth: encabezado Authorization con el **token_api crudo** (la implementación actual no exige el prefijo Bearer). Query opcional date=YYYY-MM-DD.

```
{"estado":"ok","salida":"","data":[{"id":1,"bolivares_value":36.5,"recorded_at":"..."},
  "fecha_consultada":"2026-06-01"}
```

POST/PUT/DELETE → 405 Solo lectura.

GET /api/v2/planes · /api/v2/planes/{id}

Planes / perfiles ISP

Lectura de perfiles. POST create() es alias de la lista. Filtros: incluir_deshabilitados (todos), router / id_router / id_mikrotik / nodo.

```
{"ok":true,"conteo":3,"planes":[{"...fila perfiles..."}]}
```

GET /api/v2/promociones · /activas · /{id}

Paquetes promocionales

Tablas promo_paquetes. Flags: solo_activos, solo_vigentes, id_zona / sucursal, incluir_etiquetas (default true). Si las tablas no existen: tablas_instaladas: false. Escritura → 405.

GET /api/v2/plantillasfacturacion · /{id}

Plantillas de facturación NEXO

Flags resumen e incluir_config. Devuelve plantillas[] con config e impuestos deserializados.

GET /api/v2/metrics · /tickets · /customers · /dashboard

Métricas de soporte y clientes

Solo GET. Query: mes (1-12), año (2000-2100). En tickets/dashboard: agente_id / idtecnico / idsoporte. El mes es el intervalo [día 1 00:00, día 1 del mes siguiente).

- **/metrics** — índice con rutas[].
- **/metrics/tickets** — creados, por estado, cerrados por técnico, top asuntos, interacciones del log.
- **/metrics/customers** (alias /clientes) — activos, suspendidos, retirados, nuevos del mes.
- **/metrics/dashboard** — tickets + clientes.

7.4 Hotspot

GET / **POST** /api/v2/hotspotfichas**Inventario de fichas disponibles**

Solo fichas estado = 0 (nuevas). **Nunca expone user/password**. Filtra por sucursal del operador cuando aplica.

Ruta	Función
GET/POST /hotspotfichas	Agrupado por plan
GET /por-sucursal · ?agrupar=sucursal	Por sede
GET /lista · /list	Listado paginado (limit máx. 500)
GET /{id} · /ficha?id=	Una ficha (sin credenciales)

Filtros: router, perfil, sucursal / sucursales, detalle, incluir_sucursal, grupo, user (solo filtro).

GET /api/v2/hotspotventas · /instrucciones**Datos para pagar una venta de fichas**

Sin token. Devuelve banco, código, teléfono y cédula/RIF de pago móvil. 503 si no está configurado.

POST /api/v2/hotspotventas · /pago · /c2p**Vender fichas (pago móvil o C2P)**

Tras un pago efectivo **sí devuelve user/pass** de las fichas. Opcional WhatsApp (notif_whatsapp).

Campo	Req.	Notas
token / Bearer	Sí	Operador API
id_router / router	Sí	Router hotspot
ficha_ids[] / id_ficha	Sí	Fichas a vender
referencia, monto	Sí (PM)	monto = total_bs
fecha, cedula, pm_tel, pm_banco_origen, notas, chatbot	No	
accion (en /c2p)	Sí	resumen request_otp c2p_pay

```
{"pago_efectivo":true,"id_venta":55,"fichas":[{"user":"...", "pass":"..."}], "notif_whatsapp":{}}
```

409 no conciliado. 503 módulo ventas no migrado. Efectos: marca fichas vendidas y provisiona en Mikrotik.

GET / **PUT** / **POST** /api/v2/hotspotwifi/{usuario} · /cambiar**Usuario Wi-Fi recurrente (RADIUS)**

GET consulta usuario, rate, activo, presencia en radcheck/radreply. PUT o POST /cambiar: nuevo_usuario, rate, activo: 0 (desactivar). DELETE está bloqueado por Apache: usa activo: 0.

GET /api/v2/hotspotpartido · /agenda**Agenda deportiva para portal hotspot**

Requiere token. Caché 300 s. refresh=1 ignora caché. 502 si falla la red y no hay caché.

```
{"activo":true,"partidos":[],"actualizado":"...", "fuente":"...", "desde_cache":true}
```

7.5 Operación de campo, mapa y flota

GET / **POST** /api/v2/planificaciontareas**Calendario de tareas y consulta por cédula**

Feed tipo FullCalendar. Auth token/Bearer. Requiere start y end. action: events, planning_feed, events_por_tecnico, planning_feed_operador. Si action es por técnico, operador = id login.

```
{"estado":"ok","events":[],"source":"...", "supervisor_view":true}
```

POST /consulta_cliente o /por_cedula con cedula/documento. Respuesta: cliente, tiene_planificacion, tareas[]. GET en consulta_cliente → 405.

POST /api/v2/vehiculos · /registro**Telemetría GPS / OBD**

Campo	Req.	Descripción
token	Sí	Operador API
id_dispositivo	Sí	String ≤ 100
ubicaciones[] o latitud/longitud	Sí*	Al menos una ubicación o un log
comando, parametro, unidad, formula, datetime	No	Log de sensor

201: id_log, ids_ubicacion, registrado, advertencias. 503 sin migración. 422 sin datos. GET raíz → ayuda con ejemplo.

GET /api/v2/sucursales · /activas · /{id}**Sedes visibles para el operador**

Respetar login_sucursales. Flags: detalle, incluir_zonas, incluir_routers. 404/{id} si no hay permiso. POST create() = lista. Escritura → 405.

```
{"ok":true,"sucursales":[{"id":1,"nombre":"..."}]}
```

GET /api/v2/ubicaciones · /zonas · /{id}**Zonas / ubicaciones**

Tabla zonas. Filtros: id_sucursal, detalle, incluir_sucursales. Respuesta ubicaciones[] con id, nombre, zona, id_sucursal.

GET /api/v2/nap · /coordenadas · /{id}**Cajas NAP (API NEXO)**

Por defecto solo cajas con coordenadas. Filtros: id_sucursal, solo_con_coordenadas, detalle. 403 si el operador no puede esa sucursal. cajas_nap[] con latitude/longitude parseados.

7.6 Social WiFi e IPTV

GET /api/v2/socialwifi/@usuario · ?username= · /followers**Verificar seguidor y sincronizar lista**

200 si el usuario sigue la cuenta (existe: true + data). 404 si no sigue. GET /followers sincroniza la lista remota, inserta faltantes y **elimina** a quienes dejaron de seguir.

GET / **POST** /api/v2/socialwifiregistro**Registro Social WiFi por teléfono + OTP**

Método	Ruta	Acción
GET	?telefono= · /estado	Estado del registro
POST	/ · /registro	Guardar registro
POST	/enviar_codigo	Enviar OTP
POST	/verificar_codigo	telefono + codigo/otp

GET / **POST** /api/v2/iptvsetplex**IPTV Setplex / Nora**

- GET / o /planes — lista perfilesiptv. GET /{idperfil} — un plan.
- POST /credenciales o raíz — cedula/documento → credenciales Setplex.
- POST /cambiar_plan — cedula, opcional idperfil / idservicio.

502 si NEXO actualizó pero falla Nora. PUT/DELETE → 405.

8. Webhooks (notificaciones entrantes)

Estos endpoints **los llama el banco o el proveedor**, no tu app. Debes configurar la URL en el panel del proveedor. No hay rewrite dedicado: se alcanzan como PHP bajo `/api/webhooks/`.

En el portal de documentación, crea un grupo “Webhooks” separado de “REST API”. Cada ficha debe explicar: quién llama, cómo se autentica, qué hace el servidor y qué JSON debe devolver para que el banco no reintente en loop.

WEBHOOK POST `/api/webhooks/bbva.php`

BBVA Provincial — botón de pago

JSON crudo. Campos usados: parametro4 (factura/trx), parametro2 (monto), parametro3, parametro5, estatus, nro_referencia_banco / referencia, moneda, firma. Si estatus === 0 (aprobado) registra el pago con `addPago` y actualiza `provincial_trx`. Guarda la notificación en `provincial_notifications`.

```
{"estado":"exito","salida":"...","resultados":[]}
```

No POST → 200 `{"error":"Solo se permiten peticiones POST"}`.

WEBHOOK POST `/api/webhooks/bdv.php`

Banco de Venezuela — push de pago móvil recibido

No cobra la factura. Inserta en `notificaciones_banco` y `pagos_bdv` para que luego `/api/v2/pagomovil` (u otra pantalla) concilie. Auth: lista de IPs fijas + header `x-api-key` o `api-key` = pin decodificado de una pasarela `notificacionbdv` activa.

Campo JSON	Uso
<code>referenciaBancoOrdenante</code>	Referencia
<code>numeroCliente</code>	Remitente (cédula/teléfono)
<code>monto</code>	Monto
<code>fecha (Ymd), hora (Hi o H:i:s)</code>	Momento
<code>numeroComercio, idComercio, idCliente, bancoOrdenante</code>	Contexto

Respuestas habituales HTTP 200: código "00" ok, "01" duplicado (misma referencia+remitente), "99" auth. IP no permitida → JSON de error.

WEBHOOK POST `/api/webhooks/evertec.php`

PlaceToPay / Evertec checkout

Firma del SDK PlaceToPay. Credenciales de pasarela tipo = `placetopay` y estado `on`. Responde **ok** de inmediato y procesa en segundo plano: si la referencia es un id de factura y está aprobado → `addPago`; si empieza por `SUBS_` → tokenización de tarjeta. URL de `notificationURL` en el checkout del portal.

WEBHOOK GET / POST `/api/webhooks/serdimpre.php`

Serdimpre — estado fiscal del documento

Auth: header `X-API-Key`, `Bearer`, o query `token/api_key` = ajuste `serdimpre_webhook_token`. Si `isserdimpre !== on` → 503. El id de factura llega por query, form o JSON. Consulta a Serdimpre; en etapa 4 (Facturado) puede registrar el pago.

```
{"status":"ok","code":200,"message":"...","data":{}}
```

401 no autorizado, 400 id inválido, 404 factura, 500 API Serdimpre.

Flujo de cobro (resumen)

```
BDV push --> /api/webhooks/bdv.php --> pagos_bdv
        --> /api/v2/pagomovil | pagos | pagomovilbdv --> addPago

BBVA push --> /api/webhooks/bbva.php --> addPago directo

PlaceToPay --> /api/webhooks/evertec.php --> addPago / token tarjeta

Serdimpre --> /api/webhooks/serdimpre.php --> cache fiscal + addPago opcional
```

9. Recaudadores y pasarelas bancarias

Scripts de Mikrowisp (ionCube) que consume el recaudador. Contrato oficial: POST JSON, campo token = token API del operador. Las rutas sin .php también resuelven por el rewrite del sitio.

En este servidor **sí** están Facilito, Punto Ágil, Evertec recaudador y BancoEstado. **No** están desplegados Bancard ni Dicomtecsa (aunque Mikrowisp los documenta).

POST /facilito/consultadeuda

Facilito — consultar deuda por cédula

```
{"token":"TU_TOKEN","cedula":4034567651}
```

```
{"code":"000","IDFactura":2161,
"detalle":"Pago de comprobante N° 00002161 - Vencimiento: 28/02/2022",
"valor":"500.00","mensaje":"Operación exitosa."}
```

POST /facilito/registrarpago

Facilito — registrar pago en caja

Campo	Req.	Descripción
token	Sí	Token API
IDFactura	Sí	Factura
valor	Sí	Monto
fecha	Sí	YYYY-MM-DD
secuencial	Sí	Secuencial Facilito

```
{"code":"000","autorizacion":16401,"valor":"500.00","mensaje":"Operación exitosa."}
```

POST /facilito/reversopago

Facilito — reverso

token, IDFactura, secuencial. Se usa si Facilito no recibió respuesta a tiempo o para ajuste automático.

POST /facilito/consultapago

Facilito — conciliación del día

token + fecha (YYYY-MM-DD). Devuelve el listado de { IDFactura, valor, secuencial }.

Punto Ágil usa el mismo contrato cambiando el prefijo:

Ruta	Equivalente Facilito
POST /puntoagil/consultadeuda	consulta por cédula
POST /puntoagil/registrarpago	registrar pago
POST /puntoagil/reversopago	reverso
POST /puntoagil/consultapago	pagos del día

POST /evertec/consultadeuda.php · /evertec/registrarpago.php

Evertec recaudador (red de cajas)

Distinto del webhook PlaceToPay. Archivos ionCube; el contrato lo entrega Evertec (no está en llms.txt de Mikrowisp). Trátalo como integración de vendedor: consulta deuda + registro de pago.

SOAP /bancoestado/web/?wsdl

BancoEstado Caja Vecina (Chile)

SOAP 1.x. Métodos: consultarCliente(rutCliente) y registrarPago(rutCliente, factura, monto, descripcion, pasarela, transaccion). La respuesta es XML <RESPUESTA> con ESTADO (aceptado, ...) y MENSAJE.

10. Catálogo rápido de rutas

Úsalo como índice del futuro portal (sidebar). Todas relativas a <https://TU-DOMINIO>.

Auth y proxies

Método	Ruta	Auth
POST	/api/auth	usuario/clave
POST	/api/auth?action=refresh	JWT vigente
POST	/api/v1/{Comando}	token en body
POST	/api/v11/{Comando}	Bearer
*	/api/v2/{modulo}/...	token / Bearer según módulo
*	/api/v21/{modulo}/...	Bearer JWT o token API

Comandos v1 (43)

CreateInvoice, CreateInvoiceLibre, GetInvoices, GetPlantillasFacturacion, GetInvoice, PromesaPago, PaidInvoice, DeleteInvoice, DeleteTransaccion, ReportesPago, BaseFacturas, GetPaymentLink, NewUser, ChangeFacturacionConfig, GetClientsDetails, UpdateUser, ActiveService, SuspendService, NewPreRegistro, ListInstall, GetAllClients, NewTicket, CloseTicket, ListTicket, NewSMS, GetRouters, GetMonitoreo, GetMacFromIp, GetRedesIpv4, GetIpv6FromDuid, NewService, EditService, GetCajasNap, GetOperadores, GetTareas, GetDepartamentos, CreateTarea, SmartOltGetVlans, SmartOltGetZonas, SmartOltGetProfiles, SmartOltGetODB, SmartOltAuthorizeONU.

Módulos v2 / v21

Módulo	Lectura	Escritura
consultafactura	GET deuda	—
facturacion	GET working	POST pagos
pagos	—	POST conciliación PM
pagomovil	GET/POST consulta	POST pago
pagomovilbdv	GET/POST consulta	POST validado BDV
c2p / bdvc2p / pagoc2p	resumen	OTP + débito
dolarprice, planes, promociones, plantillasfacturacion, metrics	GET	—
hotspotfichas	inventario	—
hotspotventas	instrucciones	POST venta / C2P
hotspotwifi	GET usuario	PUT/POST cambiar
hotspotpartido	agenda	—
planificacientareas	events	POST consulta_cliente
vehiculos	ayuda	POST telemetría
sucursales, ubicaciones, nap	GET	—
socialwifi	GET / followers	sync (GET)
socialwifiregistro	estado	registro + OTP
iptvsetplex	planes	credenciales / cambiar_plan

Webhooks y recaudadores

Ruta	Quién llama
POST /api/webhooks/bbva.php	BBVA Provincial
POST /api/webhooks/bdv.php	BDV (pago móvil recibido)
POST /api/webhooks/evertec.php	PlaceToPay
GET/POST /api/webhooks/serdimpre.php	Serdimpre
POST /facilito/{consultadeuda registrarpago reversopago consultapago}	Facilito
POST /puntoagil/{...}	Punto Ágil
POST /evertec/{consultadeuda registrarpago}.php	Evertec cajas

SOAP /bancoestado/web/?wsdl

BancoEstado

11. Errores frecuentes y checklist de integración

11.1 Checklist

1. Crea un usuario operador con API = 1 y genera token_api.
2. Prueba POST /api/auth y guarda el JWT.
3. Llama v21 (NEXO) o v11 (Mikrowisp) con Authorization: Bearer.
4. Para cobros VE: confirma pasarela bancodevenezuela / bdvc2p y tasa del día en dolarprice.
5. Si usas pago móvil “offline”, el banco debe pegar a /api/webhooks/bdv.php **antes** de que el cliente envíe la referencia.
6. No uses DELETE HTTP. No pongas el token en logs ni en URLs públicas.
7. Content-Type: application/json. Fechas YYYY-MM-DD. Montos con punto.

11.2 Errores que más se ven

Síntoma	Causa habitual	Qué hacer
403 Token Bearer no autorizado	token_api mal copiado o api=0	Regenerar token y habilitar API
403 ... no tiene token_api	Login JWT de un user sin token	Generar token API a ese user
409 / referencia no encontrada	El push BDV no llegó o dígitos distintos	Ver pagos_bdv; comparar últimos 6
503 pasarela inactiva	Pasarela off o sin X-API-Key	Ajustes → Pasarelas
403 en DELETE	Apache bloquea DELETE	Usar POST/PUT documentado
502 proxy	v1/v2 no respondió en localhost	Revisar Apache / ionCube / PHP
Monto no coincide	Tasa del día distinta o factura ya pagada	GET dolarprice + estado de factura

11.3 Cómo armar el portal a partir de este PDF

1. Crea un sitio (Mintlify, Redocly, ReadMe, Docusaurus o VitePress).
2. Página “Introducción” = capítulos 2–4. Página “Authentication” = capítulo 3 con Try-it de /api/auth.
3. Un grupo por familia: Mikrowisp v1, NEXO v2, Webhooks, Recaudadores.
4. Cada **Endpoint** de este PDF = una página con OpenAPI 3 (paths, requestBody, responses) como en docs.mikrosystem.net y api.smartolt.com.
5. Añade “Try it” apuntando a https://vente.rsgve.com (o el dominio del cliente) y variable {{token}}.
6. Los archivos fuente en /root/api-docs/specs/ (introducción + inventarios) sirven como markdown intermedio.

Siguiente paso sugerido. Cuando quieras el portal, pide convertir este PDF en un sitio estático con sidebar, buscador y ejemplos curl/JS/PHP por endpoint. El contrato ya está inventariado; falta solo la capa visual.

Documento generado el 6 de octubre de 2026 a partir del código desplegado en /var/www/html (API v2, webhooks, proxies) y de la referencia oficial Mikrowisp API v1.1. Sustituye TU-DOMINIO por el host real de cada instalación.